TP n°3

Architecture d'Assistants IA Locaux avec MCP

Model Context Protocol, Client/Serveur

Objectifs Pédagogiques

Durée : 3 heures
Objectif : Remplacer le parsing manuel fragile du TP précédent par le standard industriel **MCP**. Vous allez créer une architecture modulaire où un LLM interagit proprement avec votre système de fichiers.

Phase 1 : Introduction au standard MCP

Dans le TP2, nous avons vu comment créer un agent simple en Python qui utilise un LLM pour analyser des logs. Cependant, l'approche utilisée était **fragile** car elle reposait sur du parsing manuel des réponses du modèle.

Pour industrialiser ce type d'architecture, il est préférable d'utiliser un protocole standardisé pour la communication entre le LLM et les outils externes. Le **Model Context Protocol (MCP)** est un tel standard qui permet aux modèles de langage de découvrir et d'invoquer des outils de manière structurée et fiable en définissant des interfaces pour :

- **La découverte des outils :** Le LLM peut interroger le serveur MCP pour obtenir la liste des outils disponibles, leurs signatures (noms, descriptions, types d'arguments).
- **Les appels de fonctions :** Le LLM peut invoquer des outils en envoyant des messages JSON-RPC structurés, avec les arguments correctement typés.
- **Le retour des résultats :** Les résultats des appels d'outils sont renvoyés au LLM dans un format structuré.
- **La gestion des erreurs :** MCP définit des mécanismes pour gérer les erreurs lors des appels d'outils.

Description du protocole MCP Le protocole MCP utilise JSON-RPC pour la communication entre le client (interface utilisateur + LLM) et le serveur (outils). Le client envoie des requêtes JSON pour découvrir les outils et invoquer des fonctions, tandis que le serveur répond avec des messages JSON contenant les résultats ou les erreurs.

La découverte des outils se fait via une méthode spéciale `list_tools`, qui renvoie une liste d'outils avec leurs signatures. Les appels de fonctions sont effectués via la méthode `call_tool`, où le client spécifie le nom de l'outil et les arguments sous forme de dictionnaire.

Dans le détail, la structure JSON des messages MCP est la suivante :

- **Découverte des outils :** Le client envoie une requête de découverte au serveur MCP sous la forme du message donné dans le listing 1.

  Le serveur répond avec une liste d'outils disponibles (listing 2), chacun avec son nom, sa description et son schéma d'entrée.
- **Appel d'outil :** Le client envoie une requête d'appel d'outil au serveur MCP sous la forme du message donné dans le listing 3.

  Le serveur exécute l'outil et renvoie le résultat ou une erreur sous le format donné dans le listing 4.

Listing 1 Exemple de requête de découverte MCP

```
{
  "jsonrpc": "2.0",
  "method": "list_tools",
  "params": {},
  "id": 1
}
```

Listing 2 Exemple de réponse de découverte MCP

```
{
  "jsonrpc": "2.0",
  "result": {
    "tools": [
      {
        "name": "read_log_head",
        "description": "Lit les N premieres lignes d'un fichier log.",
        "inputSchema": {
          "type": "object",
          "properties": {
            "filename": { "type": "string", "description": "Nom du fichier log." },
            "lines": { "type": "integer", "description": "Nombre de lignes a lire." }
          },
          "required": ["filename"]
        }
      }
    ]
  },
  "id": 1
}
```

Listing 3 Exemple d'appel MCP

```
{
  "jsonrpc": "2.0",
  "method": "call_tool",
  "params": {
    "tool_name": "read_log_head",
    "arguments": {
      "filename": "system.log",
      "lines": 10
    }
  },
  "id": 1
}
```

Listing 4 Exemple de réponse MCP

```
{
  "jsonrpc": "2.0",
  "result": {
    "content": "Ligne 1...\nLigne 2...\n..."
  },
  "id": 1
}
```

Ce protocole est supporté par plusieurs frameworks et bibliothèques, facilitant l'intégration avec différents LLMs et environnements. Nous utiliserons ici la bibliothèque `mcp` en Python pour implémenter notre serveur MCP. Cette dernière simplifie la création de serveurs et clients MCP et la définition des outils et prends en charge la gestion des messages entre client ou serveur et le modèle de langage.

Les avantages de MCP par rapport au parsing manuel sont nombreux. En utilisant un proto-

cole standardisé, on bénéficie d'une meilleure robustesse, d'une évolutivité accrue et d'une facilité d'intégration avec divers modèles de langage. MCP permet également une gestion plus propre des erreurs et des types de données, réduisant ainsi les risques de bugs liés au parsing.

Phase 2 : Architecture Client-Serveur MCP

Dans ce TP, nous allons implémenter une architecture Client-Serveur utilisant MCP pour permettre à un modèle de langage d'interagir avec un serveur Python exposant des outils de lecture de logs.

Le modèle de langage **Gemma 3 (4B)** utilisé dans le précédent TP n'a pas été spécifiquement conçu pour MCP. Il est donc difficile de l'utiliser directement comme un client MCP natif sans avoir recours à des adaptations spécifiques telles que l'utilisation d'un moteur d'inférence bas-niveau couplé à une grammaire formelle (voir section bonus 1).

Pour simplifier, nous allons utiliser un autre modèle, `llama3.2` dans sa version *3B*, qui supporte MCP nativement via la plateforme `Ollama`.

L'architecture que nous allons mettre en place comporte 3 composants principaux. D'une part, un **Serveur MCP** écrit en Python qui expose des outils sous forme de fonctions. D'autre part, un **Client Python** qui agit comme intermédiaire entre le LLM et le serveur MCP. Enfin, le **LLM (*llama3.2 3B*)** qui interagit avec le client pour formuler des requêtes basées sur les prompts utilisateur.

De manière schématique, l'architecture représenté dans la figure 1 est la suivante :

- **Client Python :** Ce composant agit comme l'interface utilisateur et le médiateur entre le LLM (llama3.2 3B) et le serveur MCP. Il est responsable de la conversion des outils MCP au format JSON attendu par le LLM.
- **Serveur MCP :** Ce composant expose les outils de lecture de logs via le protocole MCP. Il gère les requêtes du client, exécute les fonctions demandées et renvoie les résultats.
- **LLM (llama3.2 3B) :** Le modèle de langage qui interagit avec le client pour formuler des requêtes basées sur les prompts utilisateur.

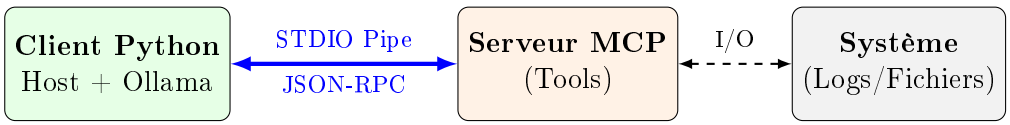

Figure 1 – Architecture Client-Serveur MCP

Dans cette architecture, le Client est générique et ne "code" pas les outils. Il demande au Serveur "Quels outils as-tu ?" et sert d'intermédiaire avec le LLM.

Étape 1 : Préparation de l'environnement

1.1 : Installation du modèle et de l'environnemnt Avant de commencer, vous devez télécharger le modèle `llama3.2:3b` sur le serveur local https://scolarite.polytech.univ-nantes.fr/cours/llm/ *(voir TP1)*. Vous devez télécharger également le fichier *ModelFile* associé qui contient le nécessaire pour activer MCP pour ce modèle. Installez ce modèle en le nommant `llama3.2:3b` à l'aide de la commande suivante :

```
ollama create llama3.2:3b -f /path/to/ModelFile
```

Enfin, ajoutez le paquet Python `mcp` à votre environnement :

```
pip install mcp
```

1.2 : Création des fichiers pour la simulation Pour simuler des ficheirs logs, nous allons utiliser un générateur simple. Pour celà exécutez le script `setup.py`. Ce script crée un dossier `logs_simulation` et y ajoute deux fichiers (`system.log` et `access.log`) contenant du texte fictif.

Le premier de ces fichiers contient des logs d'évènement système catégorisés en 5 types : *INFO*, *WARNING*, *ERROR*, *DEBUG* et *CRITICAL*.

Quant au fichier `access.log`, son contenu représente les logs d'accès à un serveur web.

Étape 2 : Création du Serveur MCP

Afin de permettre au modèle de langage d'interagir avec des outils de lecture de logs, nous allons créer un serveur MCP en Python. Ce serveur exposera des fonctions permettant de lister les fichiers de logs disponibles et de lire les premières lignes d'un fichier log spécifique.

2.1 : Fonctions à exposer Dans notre cas d'usage, nous souhaitons exposer des outils pour lire des fichiers de logs. Le serveur va donc implémenter deux fonctions principales :

- `list_logs()` : fonction qui liste les fichiers de logs disponibles dans un répertoire spécifié.
- `read_log_head(filename: str, lines: int = 10)` : fonction qui lit les `lines` premières lignes d'un fichier log donné.

Nous allons créer un serveur autonome qui expose des outils pour lire des logs.

2.2 : Mise en place du serveur La bibliothèque `mcp` fournit une classe pratique appelée `FastMCP` qui facilite la création de serveurs MCP en Python. Son fonctionnement repose sur des décorateurs pour définir les outils exposés : en préfixant la définition d'une fonction avec `@mcp.tool()`, la découverte et l'appel de cette fonction seront automatiquement gérés par le serveur MCP.

Vous pourrez de plus utiliser des annotations de type Python pour définir les schémas d'entrée des outils ainsi que la documentation de la fonction pour simplifier la génération automatique des descriptions d'outils.

Question 1 : Création du server MCP

Ajouter dans le fichier `server.py` le code nécessaire pour créer un serveur MCP selon le schéma de code défini dans le listing 5

Listing 5 serveur MCP avec `FastMCP`

```python
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("DevOps Tools")

if __name__ == "__main__":
  # Le serveur tourne en mode stdio
  mcp.run()
```

Question 2 : Découverte automatique

Modifiez le code du serveur (fichier `server.py`) pour que les outils exposés soient automatiquement découverts par le client MCP :

- Utilisez le décorateur `@mcp.tool()` pour exposer les fonctions.
- Fournissez des descriptions claires pour chaque outil en utilisant les docstrings.
- Utilisez les annotations de type Python pour définir les schémas d'entrée des outils.

Étape 3 : Création du Client MCP

Le LLM ne peut pas communiquer directement avec le serveur MCP. Nous devons donc créer un client Python qui servira d'intermédiaire entre le LLM et le serveur MCP. Ce client sera responsable de la découverte des outils disponibles sur le serveur MCP, de la conversion de ces outils au format JSON attendu par le LLM, et de la gestion des appels d'outils basés sur les réponses du LLM.

Dans notre cas d'utilisation, nous allons lancer le server MCP en mode **stdio** (entrée/sortie standard) et connecter le client directement à ce serveur via un pipe STDIO. Pour simplifier le lancement du serveur et la connexion du client au serveur, nous utiliserons le fichier `launcher.py`. Vous n'avez pas besoin de modifier ce fichier, il se charge de lancer le serveur MCP en arrière-plan et de connecter le client via un pipe STDIO.

Le code du client sera écrit dans le fichier `client.py`.

3.1 : Découverte des outils La première tâche du client est de découvrir les outils disponibles sur le serveur MCP. Pour cela, nous allons utiliser la méthode `list_tools()` fournie par la bibliothèque `mcp`. Cette méthode envoie une requête de découverte au serveur MCP et récupère la liste des outils exposés.

Question 3 : Implémentation de la découverte

Implémentez dans le fichier `client.py` la découverte des outils disponibles sur le serveur MCP en vous basant sur le code donné dans le listing 6.

Listing 6 Découverte des outils MCP en Python

```
tools_list = await session.list_tools()
```

Par la suite, il est nécessaire de convertir les outils découverts au format JSON attendu par le LLM. Chaque outil doit être représenté sous la forme d'un dictionnaire JSON contenant son nom, sa description et son schéma d'entrée selon l'exemple donné dans le listing 7.

Listing 7 Conversion des outils MCP au format JSON

```
{
  'type': 'function',
  'function': {
    'name': nom_de_l_outil (tool.name),
    'description': description_de_l_outil (tool.description),
    'parameters': schema_d_entree (tool.inputSchema)
  }
}
```

Question 4 : Conversion des outils au format JSON d'Ollama

Implémentez dans le fichier `client.py` la conversion des outils découverts au format JSON attendu par le LLM en vous basant sur le code donné dans le listing **??**.

3.2 : La boucle interactive (Chat Loop) La boucle interactive est le cœur du client MCP. Elle permet à l'utilisateur de saisir des prompts, d'envoyer ces prompts au LLM, de gérer les appels d'outils basés sur les réponses du LLM, et de renvoyer les résultats au LLM pour obtenir une réponse finale.

Elle fonctionne de la manière suivante :

1. L'utilisateur saisit un prompt.

2. Le client envoie le prompt au LLM avec la liste des outils disponibles.
3. Le LLM analyse le prompt et décide s'il doit appeler un outil.
4. Si un outil est appelé, le client exécute l'outil via le serveur MCP et récupère le résultat.
5. Le client ajoute le résultat de l'outil à l'historique des messages et renvoie une nouvelle requête au LLM pour obtenir une réponse finale.
6. Le LLM renvoie la réponse finale au client, qui l'affiche à l'utilisateur.
7. Le processus se répète jusqu'à ce que l'utilisateur décide de quitter.

La figure 2 illustre le flux MCP et la boucle interactive.

Question 5 : Examen du code

Examinez le code de la boucle interactive donné dans le fichier `client.py` et expliquez son fonctionnement.

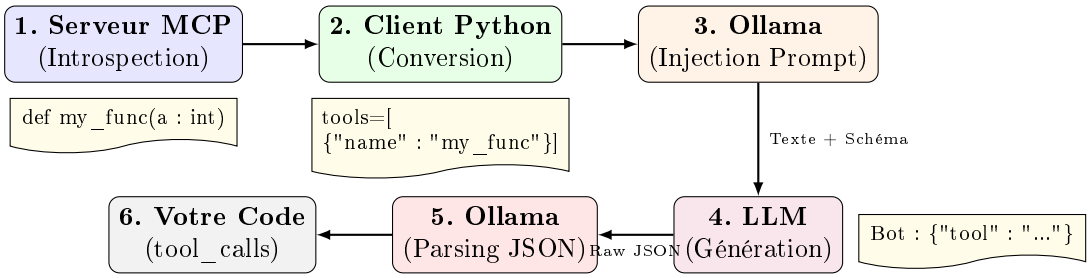

Figure 2 – Le voyage de la donnée : De la fonction Python au Tool Call

Phase 3 : Tests et Analyse

Étape 4 : Scénario de test

Question 6 : Premiers tests

Lancez le client et jouez le rôle du DevOps en posant par exemple les quesitons suivantes :

1. Quels sont les fichiers de logs disponibles ?
2. S'il y a un fichier 'system.log', lis les 5 premières lignes et dis-moi s'il y a des erreurs."

Question 7 : Amélioration du prompt système

Modifiez le prompt système dans le fichier `client.py` pour orienter le modèle vers l'utilisation du protocole MCP en décommentnt le message pré-préparé.

Question 8 : Analyse

Comparez la fiabilité de ce système avec l'agent manuel du TP 2.

1. Avez-vous dû écrire une Regex pour parser la réponse ? (Non, JSON natif).
2. Le schéma des outils (`inputSchema`) assure-t-il que le LLM envoie les bons types d'arguments (int vs str) ?

Étape 5 : Utilisation d'un autre serveur MCP

Question 9 : Réutilisation de la classe `SmartHome`

Pour montrer la modularité de MCP, réutilisez la classe `SmartHome` du TP précédent (fichier `smart_home.py`) en tant que serveur MCP. Adaptez-la pour exposer des outils MCP similaires à ceux utilisés précédemment.

Question 10 : Utilisation d'un MCP public

Utilisez le serveur public d'interrogation spatiale de Google : https://developers.google.com/maps/ai/grounding-lite/reference/mcp/search_places

Bonus 1 : du "Probabiliste" au "Déterministe"

Dans ce TP, nous avons utilisé `Ollama` et le *Prompt Engineering*. Comme vous l'avez constaté, c'est une approche **probabiliste** : nous demandons poliment au modèle de générer du JSON, et nous espérons qu'il respectera la syntaxe. Cependant, le modèle conserve toujours une probabilité non-nulle de générer du texte superflu ou d'oublier une accolade.

Pour passer en **production**, on utilise une approche différente appelée la **Génération Contrainte** (ou *Grammars*).

1. Le principe : "Verrouiller le clavier"

Au lieu de laisser le modèle choisir librement le token suivant parmi tout son vocabulaire, on utilise un moteur d'inférence bas niveau (comme `llama.cpp`) couplé à une grammaire formelle (GBNF pour le cas de `llama.cpp`).

Le fonctionnement est mathématique :

- Vous fournissez le schéma exact de la réponse attendue.
- Avant de générer un token, le moteur consulte la grammaire.
- Si le modèle souhaite générer un mot interdit par la grammaire (ex : "Bonjour" alors qu'une accolade `{` est attendue), le moteur force la probabilité de ce token à **0**.

Le modèle est alors *physiquement obligé* de générer une sortie syntaxiquement valide.

2. L'Architecture "Pensée-Action"

Pour conserver un Agent capable de raisonner et d'agir sans briser la structure, on impose une structure JSON rigide qui encapsule les deux aspects comme indiqué dans le listing 8.

Listing 8 Structure JSON "Pensée-Action"

```
{
  "thought": "L'utilisateur veut voir les logs. Je dois utiliser l'outil...",
  "message_to_user": "Je vais regarder le fichier system.log pour vous.",
  "tool_call": {
    "name": "read_log",
    "arguments": { "filename": "system.log" }
  }
}
```

Cette méthode garantit une fiabilité de **100%** sur la syntaxe, transformant un prototype instable en un outil industriel robuste.